Análisis de complejidad computacional: `ParallelARIMAWorkflow` (TSLib / Spark) frente a ARIMA lineal (statsmodels)

Notas técnicas (TSLib)

23 de abril de 2026

Resumen

Se formaliza el coste asintótico del flujo de once pasos implementado en `ParallelARIMAWorkflow` frente a un ajuste ARIMA clásico en un solo proceso. Se derivan desigualdades de *break-even* en tiempo de pared, se ilustra con un ejemplo numérico, una tabla de **calibración medida** (script Python que rellena macros en `parallel_arima_complexity_calibration.tex`) y el enlace a métodos en `parallel_arima_workflow.py` y a statsmodels en Shiny.

Índice

1. Notación y variables (alineación con el código) **2**
1.1. Tamaño de la rejilla de órdenes 2

2. Implementación en código (TSLib) **2**

3. Modelo de coste de un único ajuste MLE **2**

4. Desglose por pasos del `ParallelARIMAWorkflow` **3**
4.1. Tiempo de pared agregado (modelo simplificado) 4

5. Baseline lineal (statsmodels) **4**
5.1. Un solo modelo en la serie completa 4
5.2. Búsqueda en rejilla secuencial (comparable al espacio de órdenes) 4

6. Desigualdades de break-even **4**
6.1. Frente a un único ajuste lineal 4
6.2. Frente a búsqueda en rejilla secuencial en n 5

7. Estimación con umbral numérico: cuándo gana el paralelo **5**
7.1. Modelo de tiempos medidos 5
7.2. Umbral en tiempo de un fit completo 6
7.3. Umbral en longitud de serie n^* (modelo potencia) 6
7.4. Frente a un solo fit lineal 6

8. Calibración medida y regeneración automática **7**
8.1. Comando para volver a medir y actualizar el PDF 7
8.2. Resultados insertados en este documento 7

9. Ejemplo numérico aplicado **7**

10. Conclusión **8**

1. Notación y variables (alineación con el código)

Sea la serie observada de longitud n. Tras el paso de estacionariedad (log y/o diferencias hasta `d_max`), se obtiene la serie de trabajo de longitud efectiva que denotamos también por n salvo que se pierdan observaciones por diferenciación (en ese caso, sustitúyase n por n_{eff}).

Cuadro 1: Parámetros simbólicos y correspondencia con TSLib / PySpark.

Símbolo	Origen típico en código	Interpretación
n	`len(working_data_)`	Longitud de la serie de trabajo
d	orden de integración elegido en paso 1	Fijo para la rejilla (p, d, q)
$P_{\text{máx}}$	`max_p`	Cota superior de orden AR en la rejilla
$Q_{\text{máx}}$	`max_q`	Cota superior de orden MA en la rejilla
W_s	`num_sliding_windows`	Número de ventanas deslizantes
W_f	`num_fixed_windows`	Número de ventanas fijas (validación)
K	`full_sample_reconcile_top_k`	Ajustes en muestra completa (top-K)
P	paralelismo efectivo Spark	Número de tareas útiles concurrentes (aprox.)
α	overhead JVM / plan / shuffle / I/O	Término fijo por *jobs* Spark
L	longitud media ventana deslizante	$\Theta(n)$ o $\Theta(n/W_s)$ según diseño
L_f	longitud típica ventana fija	Acotada por n

1.1. Tamaño de la rejilla de órdenes

Tras fijar d, el generador `_generate_parameter_combinations` recorre $p \in \{0, \ldots, P_{\text{máx}}\}$ y $q \in \{0, \ldots, Q_{\text{máx}}\}$ y excluye $(p, q) = (0, 0)$. Definimos

$$C \;:=\; (P_{\text{máx}} + 1)(Q_{\text{máx}} + 1) - 1. \tag{1}$$

2. Implementación en código (TSLib)

La clase `ParallelARIMAWorkflow` vive en `tslib/spark/parallel_arima_workflow.py`. El método público `fit` encadena los pasos 1–10; `predict` es el paso 11 (pronóstico con el modelo final, sin Spark). La Tabla 2 resume la correspondencia entre la demostración de complejidad y las rutinas internas (nombres reales del repositorio).

El término α de la ecuación (5) incluye tiempos medidos en `spark_timing_` (p. ej. `executor_warmup_s`) y el coste de materializar/reparticionar el DataFrame de tareas antes del `mapInPandas`. La comparación lineal en la aplicación Shiny usa `statsmodels ARIMA` sobre la misma `working_data_`; véase `tslib-shiny-app/services/tslib_service.py` y `docs/SPARK_VS_LINEAR.md`.

3. Modelo de coste de un único ajuste MLE

Denotemos por $f(m; p, d, q)$ el tiempo (o FLOPs proporcionales) de ajustar un ARIMA(p, d, q) por máxima verosimilitud sobre una subserie de longitud m (incluye iteraciones del optimizador). Supondremos que existe $\gamma \geq 1$ y constantes $c_1, c_2 > 0$ tales que

$$c_1\, m^{\gamma} \;\leq\; f(m; p, d, q) \;\leq\; c_2\, m^{\gamma}\, h(p, q) \tag{2}$$

para un factor $h(p, q)$ acotado polinomialmente en (p, q) en la rejilla. En la práctica, statsmodels y `ARIMAProcess` pueden estar entre $\gamma \approx 1$ (idealización lineal en m) y $\gamma > 1$ si el coste por iteración crece superlinealmente con m.

Cuadro 2: Pasos del workflow → métodos en `parallel_arima_workflow.py`.

Paso	Rol en el análisis T_{par}	Método(s) principal(es)
1	$\mathcal{O}(n \cdot d_{\text{máx}})$ ADF/KPSS, log opcional	`_determine_differencing_order`
2–3	$\mathcal{O}(n)$ ACF/PACF; $\mathcal{O}(C)$ rejilla	`_determine_parameter_ranges`, `_apply_grid_mode_to_config`, `_generate_parameter_combinations`
4	W_s ventanas, longitud $\sim L$	`_create_sliding_windows`
5	$\Theta(W_s C\, f(L))$ repartido en P	`_fit_models_parallel_sliding` (`mapInPandas` / DataFrame de tareas)
6	agregación $\mathcal{O}(W_s C)$	`_select_global_model`
6b	$K\, f(n)$ opcional	`_reconcile_order_full_sample_aic`
7–8	$\Theta(W_f\, f(L_f))$	`_create_fixed_windows`, `_backtest_fixed_windows`
9	diagnósticos por ventana fija	`_diagnose_residuals`
10	$\mathcal{O}(f(n))$ vecinos $(p \pm 1, q \pm 1)$	`_check_diagnostic_failures`, `_try_local_adjustment`
cierre	$f(n)$ ajuste final	`ARIMAProcess(...).fit(working_data)` en `fit`
11	pronóstico	`predict` → `ARIMAProcess.predict`

Observación 1. *El análisis de* break-even *solo usa órdenes de magnitud; las constantes c_1, c_2 se absorben en calibración empírica.*

4. Desglose por pasos del `ParallelARIMAWorkflow`

El método `fit` orquesta los pasos 1–10; el paso 11 es `predict` (no paralelizado). A nivel de complejidad:

Paso 1 – Diferenciación / estacionariedad Bucle ADF/KPSS hasta `d_max`: coste del orden $\mathcal{O}(n \cdot d_{\text{máx}})$ en el driver.

Pasos 2–3 – Rejilla (p, d, q) Generación de C tuplas: $\mathcal{O}(C)$. ACF/PACF sobre n: $\mathcal{O}(n)$ (driver).

Paso 4 – Ventanas deslizantes Construcción de W_s ventanas: $\mathcal{O}(W_s \cdot L)$; con ventanas que particionan o solapan una fracción fija de n, suele ser $\mathcal{O}(nW_s)$ en el peor caso razonable o $\mathcal{O}(n)$ si $L = \Theta(n/W_s)$.

Paso 5 – Spark: MLE por (ventana, orden) Se lanzan del orden de $W_s \cdot C$ ajustes independientes sobre longitud $\approx L$. Coste serial equivalente:

$$T_{\text{ser}}^{(5)} \;=\; \Theta\big(W_s\, C\, f(L)\big). \tag{3}$$

Tiempo de pared ideal (paralelismo perfecto, sin overhead): $T_{\text{par}}^{(5)} \approx T_{\text{ser}}^{(5)}/P$.

Paso 6 – Selección global Agregación sobre resultados: $\mathcal{O}(W_s \cdot C)$ (dominado por el paso 5).

Paso 6b – Reconciliación opcional K ajustes en muestra completa:

$$T^{(6b)} \;=\; \Theta\big(K\, f(n)\big). \tag{4}$$

Pasos 7–8 – Ventanas fijas y backtest $\Theta(W_f\, f(L_f))$ más overhead Spark si aplica.

Paso 9 – Diagnósticos $\mathcal{O}(W_f)$ veces coste de diagnóstico sobre residuos; usualmente menor que el paso 5 si $W_f \ll W_s C$.

Paso 10 – Ajuste local en vecinos Acotado por un número constante de órdenes adyacentes: $\mathcal{O}(f(n))$ en el peor caso.

Cierre – Modelo final Un ajuste `ARIMAProcess.fit` en toda la serie: $\Theta(f(n))$.

4.1. Tiempo de pared agregado (modelo simplificado)

Agrupando términos y añadiendo overhead Spark α:

$$T_{\text{par}} \;\approx\; \alpha \;+\; \frac{W_s\, C}{P}\, f(L) \;+\; K\, f(n) \;+\; \mathcal{O}\big(W_f\, f(L_f)\big) \;+\; f(n). \tag{5}$$

5. Baseline lineal (statsmodels)

5.1. Un solo modelo en la serie completa

Si el baseline es un único `ARIMA(p,d,q).fit` sobre longitud n:

$$T_{\text{lin}}^{(1)} \;=\; \Theta\big(f(n)\big). \tag{6}$$

5.2. Búsqueda en rejilla secuencial (comparable al espacio de órdenes)

Si el baseline recorre los mismos C órdenes, cada uno en longitud n, de forma secuencial:

$$T_{\text{lin}}^{(C)} \;=\; \Theta\big(C\, f(n)\big). \tag{7}$$

6. Desigualdades de break-even

6.1. Frente a un único ajuste lineal

Queremos $T_{\text{par}} < T_{\text{lin}}^{(1)}$. Sustituyendo (5) y (6) y conservando términos dominantes:

$$\alpha + \frac{W_s C}{P} f(L) + K f(n) + f(n) \;<\; c\, f(n) \tag{8}$$

para alguna constante c que encapsula el baseline. Como $K f(n) + f(n) = \Theta(f(n))$ con $K \geq 1$ típicamente, la parte izquierda contiene ya un término lineal en $f(n)$ *más* el término positivo $\frac{W_s C}{P} f(L)$. Por tanto, salvo que $\frac{W_s C}{P} f(L)$ y α sean despreciables frente a $f(n)$ (o $f(n)$ sea artificialmente grande), **no existe ventaja asintótica** del workflow paralelo frente a *un solo* fit en n: el pipeline está diseñado para exploración robusta (muchas ventanas y órdenes), no para minimizar trabajo total frente a un único MLE.

Observación 2 (Lectura práctica)**.** *Coincide con la nota de repositorio* `SPARK_VS_LINEAR.md`*: con una sola serie y rejilla moderada, el coste fijo α y la multiplicidad $W_s C$ suelen hacer que el lineal gane si el objetivo es solo "un ARIMA en n".*

6.2. Frente a búsqueda en rejilla secuencial en n

Comparamos (5) con (7). Idealizando $f(m) = c\,m^\gamma$ con $\gamma \geq 1$ y omitiendo W_f, α temporalmente:

$$\frac{W_s C}{P} L^\gamma \;<\; C\,n^\gamma \quad\Longleftrightarrow\quad \frac{W_s}{P}\left(\frac{L}{n}\right)^\gamma \;<\; 1. \tag{9}$$

Si $L \approx n/W_s$ (ventanas que cubren en promedio una fracción $1/W_s$ de la serie):

$$\frac{W_s}{P} W_s^{-\gamma} \;<\; 1 \quad\Longleftrightarrow\quad W_s^{1-\gamma} \;<\; P. \tag{10}$$

- Si $\gamma = 1$: (10) se reduce a $1 < P$: cualquier paralelismo $P > 1$ reduce el tiempo de pared del bloque paralelo respecto a la ejecución secuencial *de ese bloque*; el trabajo total serial equivalente del paso 5 es del mismo orden $\Theta(Cn)$ que C fits completos lineales en n bajo $L \approx n/W_s$ y f lineal en m.
- Si $\gamma > 1$: al crecer W_s, el miembro izquierdo $W_s^{1-\gamma} = W_s^{-(\gamma-1)}$ decrece: **muchas ventanas cortas** reducen el coste agregado del paso 5 frente a C fits en longitud n, y el paralelismo P acota el tiempo de pared.

Restaurando overhead, una condición suficiente grosera para que el paralelo gane frente a la rejilla secuencial completa es

$$\alpha + \frac{W_s C}{P} L^\gamma + (K+1)n^\gamma \;<\; C\,n^\gamma, \tag{11}$$

es decir,

$$\boxed{\alpha \;<\; C\,n^\gamma\left(1 - \frac{W_s}{P}\left(\frac{L}{n}\right)^\gamma\right) - (K+1)n^\gamma} \tag{12}$$

siempre que el paréntesis sea positivo. En la práctica se calibra f y α por medición.

7. Estimación con umbral numérico: cuándo gana el paralelo

Sí se puede obtener un **número duro** si fijamos un modelo de tiempos medibles y resolvemos la igualdad en el *break-even*. Las magnitudes deben obtenerse en tu máquina o cluster (cronómetro o `spark_timing_` + profiling del paso 5).

7.1. Modelo de tiempos medidos

Sea τ_{task} el tiempo medio de una tarea del paso 5 (un ajuste MLE en una ventana deslizante de longitud típica L). Sea $\tau_{\text{full}}(n)$ el tiempo de un ajuste MLE en toda la serie de longitud n (mismo orden promedio que en la rejilla lineal). Sea α el overhead acumulado Spark relevante (JVM, repartición, materialización). Ignorando por simplicidad el backtest frente a τ_{full}, una comparación honesta frente a **rejilla lineal completa** en n es

$$T_{\text{par}} \approx \alpha + \frac{W_s\,C}{P}\,\tau_{\text{task}} + (K+1)\,\tau_{\text{full}}(n), \qquad T_{\text{lin}}^{(C)} \approx C\,\tau_{\text{full}}(n). \tag{13}$$

El paralelo gana si $T_{\text{par}} < T_{\text{lin}}^{(C)}$, es decir

$$\boxed{\alpha + \frac{W_s\,C}{P}\,\tau_{\text{task}} \;<\; \big(C - K - 1\big)\,\tau_{\text{full}}(n)} \tag{14}$$

siempre que $C > K+1$ (en la práctica sí: p. ej. $C = 35$, $K = 15$ da $35 > 16$). Si $C \leq K+1$, la reconciliación domina y la desigualdad no tiene sentido físico para comparar contra C fits únicos.

7.2. Umbral en tiempo de un fit completo

Despejando $\tau_{\text{full}}(n)$ en la igualdad $T_{\text{par}} = T_{\text{lin}}^{(C)}$:

$$\tau_{\text{full}}^{*}(n) \;=\; \frac{\alpha + \dfrac{W_s\,C}{P}\,\tau_{\text{task}}}{C-K-1}. \tag{15}$$

Si en la práctica $\tau_{\text{full}}(n) > \tau_{\text{full}}^{*}$ (fits lentos o n grande), el workflow paralelo **supera** a recorrer los C modelos en serie sobre longitud n. Si τ_{full} es muy pequeño (series cortas o optimizador muy rápido), gana el lineal.

Ejemplo con cifras fijas. Con $W_s = 8$, $C = 35$, $P = 8$, $K = 15$ se tiene $\frac{W_s C}{P} = 35$ y $C - K - 1 = 19$. Si $\alpha = 3\,\text{s}$ y $\tau_{\text{task}} = 0{,}05\,\text{s}$,

$$\tau_{\text{full}}^{*} = \frac{3 + 35\cdot 0{,}05}{19} = \frac{4{,}75}{19} \approx \mathbf{0{,}25}\,\text{s}.$$

Cada fit lineal en muestra completa debe durar *menos* de unos 0,25 s para que la rejilla secuencial de 35 órdenes gane; si cada fit tarda 0,8 s, el paralelo gana con margen ($T_{\text{lin}}^{(C)} \approx 28\,\text{s}$ frente a $T_{\text{par}} \approx 3 + 1{,}75 + 16\cdot 0{,}8 \approx 17{,}55\,\text{s}$).

7.3. Umbral en longitud de serie n^* (modelo potencia)

Supongamos $\tau_{\text{full}}(n) = c\,n^{\gamma}$ y $\tau_{\text{task}}(n) = c\,(n/W_s)^{\gamma}$ (ventanas homogéneas de longitud n/W_s). Sustituyendo en (14) y despejando n en el caso borde $\alpha + \frac{W_s C}{P}\,c\,(n/W_s)^{\gamma} = (C-K-1)\,c\,n^{\gamma}$:

$$\boxed{n^{*} \;=\; \left(\frac{\alpha}{c\left(C-K-1-\dfrac{C}{P}\,W_s^{1-\gamma}\right)}\right)^{1/\gamma}}. \tag{16}$$

Para $\gamma = 1$ la fórmula se simplifica a

$$n^{*} \;=\; \frac{\alpha}{c\left(C-K-1-\dfrac{C}{P}\right)} \qquad \text{(requiere } C-K-1-\tfrac{C}{P} > 0\text{)}. \tag{17}$$

Con los mismos $C = 35$, $K = 15$, $P = 8$: $C - K - 1 - C/P = 35 - 16 - 4{,}375 = 14{,}625$. Si $\alpha = 6\,\text{s}$ y se estima $c = \tau_{\text{full}}/n$ en una corrida piloto con $n_0 = 2000$ y $\tau_{\text{full}} = 1{,}2\,\text{s}$ (luego $c \approx 6\times 10^{-4}\,\text{s}$ por observación),

$$n^{*} \approx \frac{6}{6\times 10^{-4}\times 14{,}625} \approx \mathbf{684} \text{ observaciones.}$$

Interpretación: frente a **una rejilla secuencial de C fits en longitud** n, el modelo predice que, con esos α y c, para $n \gtrsim 700$ el paralelo ya compensa el overhead; para n menores, el lineal puede ser más rápido. El número cambia al cambiar α, P, K o la calibración de c.

7.4. Frente a un solo fit lineal

Para $T_{\text{lin}}^{(1)} \approx c\,n^{\gamma}$ la desigualdad $T_{\text{par}} < T_{\text{lin}}^{(1)}$ exige $\alpha + \frac{W_s C}{P} c(n/W_s)^{\gamma} + (K+1)cn^{\gamma} < cn^{\gamma}$, es decir $\alpha + \frac{W_s C}{P} c(n/W_s)^{\gamma} < -Kcn^{\gamma}$, imposible si $K \geq 1$ y todos los términos son positivos. En este modelo **no hay** n^* finito: el workflow no está optimizado para batir a *un único* MLE; el umbral (16) aplica frente a **búsqueda comparable** en C órdenes.

8. Calibración medida y regeneración automática

Los valores numéricos de esta sección provienen del fichero `parallel_arima_complexity_calibration.tex`, generado por el script `docs/calibrate_parallel_arima_complexity.py` (última ejecución: `2026-04-23 15:53 CST`). El script mide, sobre una serie sintética acumulativa Gaussiana de longitud 2000 y una ventana de longitud 250, la mediana de tiempos de `ARIMAProcess`(2,0,2) con `n_jobs=1` (mismo núcleo MLE que en el paso distribuido del workflow). Para α, intenta PySpark en `local[1]` (una sesión y un `count` mínimo); si PySpark no está instalado, $\alpha = 0$ y el umbral n^* del modelo lineal no aplica.

8.1. Comando para volver a medir y actualizar el PDF

Desde la raíz del paquete `time-series-library` (con un entorno virtual que cumpla `requirements.txt`):

```
python3 -m venv .venv-doc-calib
.venv-doc-calib/bin/pip install -r requirements.txt
.venv-doc-calib/bin/python docs/calibrate_parallel_arima_complexity.py
cd docs && pdflatex parallel_arima_complexity_analysis.tex
pdflatex parallel_arima_complexity_analysis.tex
```

8.2. Resultados insertados en este documento

Los parámetros de la desigualdad (14) se fijan como en el texto ($W_s = 8$, $C = 35$, $P = 8$, $K = 15$). La Tabla 3 resume las magnitudes medidas.

Cuadro 3: Calibración local (TSLib `ARIMAProcess`, mediana de varias corridas).

Magnitud	Valor
α (overhead Spark; 0 si no hay PySpark)	0,0000 s
τ_{task} (fit en ventana de longitud 250)	0,4231 s
τ_{full} (fit en serie de longitud 2000)	1,0189 s
$c = \tau_{\text{full}}/n$ (modelo $\gamma = 1$)	0,000509 s/obs.
τ^*_{full} [ec. (15)]	0,7794 s
T_{par} / $T^{(C)}_{\text{lin}}$ (estimadores (13))	31,112 s / 35,663 s

Con $\alpha \approx 0$ (sin PySpark en esta corrida), la fórmula $n^* = \alpha/(c\,(C - K - 1 - C/P))$ se anula: el cuello pasa a ser solo la relación τ_{task} vs τ_{full} y el término $(K{+}1)\tau_{\text{full}}$ en (13). Para obtener un n^* numérico de overhead, instala Java+PySpark y vuelve a ejecutar el script.

9. Ejemplo numérico aplicado

Se toman valores inspirados en `_determine_parameter_ranges` para series "medianas" ($500 < n \leq 2000$) en `parallel_arima_workflow.py`: $P_{\text{máx}} = Q_{\text{máx}} = 5$, de modo que $C = (6)(6) - 1 = 35$. Sea $n = 1500$, $W_s = 8$, $W_f = 7$, $K = 15$, $P = 8$ ejecutores útiles. Suponga $L = 300$ (cada ventana deslizante usa unos 300 puntos) y $\gamma = 1{,}2$, $c = 1$ en unidades arbitrarias de tiempo por punto elevado a γ (solo para ilustrar órdenes relativos).

$$\frac{W_s C}{P}\, L^\gamma = \frac{8 \cdot 35}{8} \cdot 300^{1,2} \approx 35 \cdot 300^{1,2},$$
$$C\, n^\gamma = 35 \cdot 1500^{1,2}.$$

Numéricamente, $300^{1,2} \approx 623$, $1500^{1,2} \approx 5766$, luego

$$\frac{W_s C}{P} L^\gamma \approx 35 \times 623 \approx 2{,}18 \times 10^4,$$
$$C\, n^\gamma \approx 35 \times 5766 \approx 2{,}02 \times 10^5.$$

El cociente $\dfrac{W_s C\, L^\gamma / P}{C\, n^\gamma} = \dfrac{W_s\, L^\gamma}{P\, n^\gamma} \approx 0{,}11$: el bloque paralelo idealizado del paso 5 sería del orden de *un 11 %* del coste de ajustar los C modelos en serie completa bajo esta hipótesis de f. Si $\alpha \approx 2 \times 10^4$ (segundos escalados, p. ej. arranque Spark), entonces α es comparable al numerador anterior: el overhead puede *anular* la ventaja si n no es lo bastante grande.

Con $\gamma = 1$ y $L = n/W_s = 1500/8 \approx 187{,}5$:

$$\frac{W_s C}{P} L = \frac{8 \cdot 35}{8} \cdot 187{,}5 = 35 \cdot 187{,}5 = 6562{,}5, \qquad Cn = 35 \cdot 1500 = 52500.$$

El paso 5 tiene trabajo serial equivalente $\approx 6{,}56 \times 10^3$ frente a $5{,}25 \times 10^4$ para 35 fits en n: cociente $\approx 0{,}125 = 1/W_s$, coherente con $W_s \cdot (n/W_s) = n$ por combinación cuando f es lineal en m.

10. Conclusión

El desglose (5) separa **overhead** (α), **paralelismo** (P), **multiplicidad de tareas** ($W_s C$), **longitud de ventana** (L) y **reconciliación** ($Kf(n)$). Para **rejilla lineal comparable**, la sección 7 da umbrales explícitos (15)–(17) en función de α, τ_{task}, τ_{full} y c. Frente a *un solo* fit (6), no se espera cruce finito en el modelo con $K \geq 1$; conviene medir y reportar τ^*_{full} y n^* calibrados al entorno (sección 8).

Referencias

[1] TSLib: `tslib/spark/parallel_arima_workflow.py`, `docs/SPARK_VS_LINEAR.md`.